knowledge base, increasing systematic exploratory behavior. The active column in the task-agnostic phase then utilizes this knowledge base, further maximizing its intrinsic reward based on the systematic exploratory behavior. This acts as a pre-training phase before downstream tasks are applied.

It has been shown that the most general variation of the task-agnostic phase improves performance by accelerating transfer learning in the progress phase for downstream tasks, surpassing all three continual learning baselines, including the Progress and Compress method, in the Atari domain. Consequently, the Task-Agnostic Policy Distillation framework has demonstrated promising results in enhancing positive forward transfer and learning in scenarios without clear task boundaries.

Our approach addresses catastrophic forgetting by using Elastic Weight Consolidation (EWC) to protect old knowledge. Positive forward transfer is achieved through lateral connections from the knowledge base to the active column. Scalability across tasks is ensured by utilizing a single policy network for retaining old knowledge of previous tasks. This makes our framework well-suited to environments with restricted memory and onboard resources. Learning without clear task boundaries is facilitated by a task-agnostic phase that encourages exploration without relying on task-specific extrinsic rewards.

While the obtained results are promising, there is potential for further improvement. This includes exploring new variations of the task-agnostic phase. It would also be valuable to analyze variations within other domains. For instance, applying the general Variation 1 to domains such as robotic tasks. Furthermore, incorporating the intrinsic reward during the progress phase on a downstream task allows for optimizing both extrinsic and intrinsic rewards in an alternating fashion. This approach would likely be especially advantageous for long-horizon robotic tasks, where balancing exploration and exploitation is challenging, as rewards for exploration may not be received immediately.

Declarations

Conflicts of interest. The authors have no relevant financial or non-financial interests to disclose.

Data availability statement. The datasets used and/or analysed during the current study available from the corresponding author on reasonable request.

Code availability statement. The implementation of our TAPD algorithm, along with the baseline methods and the code to reproduce our experiments, is available at the following repository: https://github.com/wabbajack1/TAPD.

Appendix A Progress and Compress Phases

Progress. The progress phase is designed to effectively minimize the loss of a specified task, i.e., $\mathcal{L}^{progress} = -\mathbb{E}_\pi \left[\sum_{n=0}^{\infty} \gamma^n r_{t+n}\right]$, which is equivalent to the RL objective $\max_\pi \mathbb{E}_\pi \left[\sum_{n=0}^{\infty} \gamma^n r_{t+n}\right]$, where r is the reward and γ is the discount factor. As in Schwarz et al. [13], we used a distributed variant of the actor-critic architecture (A2C) to learn both the policy $\pi(a_t|s_t;\theta_1)$ and the value function $V(s_t;\theta_2)$, from raw pixels,

within both phases. The policy and value function (both referred to as *active column*) share a convolutional encoder, similar to the approach described by Mnih et al. [37].

Compress. The compress phase incorporates a variety of methods, making the implementation and fine-tuning of hyperparameters the most time-intensive aspect of this process. Unlike the progress phase, during which the primary focus is on one task, the compress phase is crucial for retaining knowledge as tasks are encountered sequentially. After acquiring knowledge for a designated number of timesteps during the progress phase, the most recent policy (the active column) of the current task is distilled into the knowledge base network (the same copy of the network architecture was used as in the progress phase) by minimizing the following distillation loss with respect to the parameters θ_{kb} of the knowledge base:

$$\mathcal{L}^{Distill}(\theta_{kb}) = \mathbb{E}\left[D_{\mathrm{KL}}\left(\pi_k(\cdot|x;\theta_{active}) \parallel \pi_{kb}(\cdot|x;\theta_{kb})\right)\right] + \frac{\lambda}{2}\gamma F_{k-1}\left\|\theta_{kb} - \theta_{kb}^{*(k-1)}\right\|_2^2 \tag{A1}$$

where x is the input and $\pi_k(\cdot|x;\theta_{active})$, $\pi_{kb}(\cdot|x;\theta_{kb})$ are the policies of the active column (after learning on task k) and the knowledge base, respectively. F_{k-1} is the diagonal Fisher Information Matrix (FIM) of the previous tasks and $\gamma > 0$ is the forgetting constant, introducing gracefully forgetting old knowledge (mind the overloaded notation γ). λ is the importance of the penalty term. $\theta_{kb}^{*(k-1)}$ are the optimal parameters of the previous tasks. The loss function $\mathcal{L}^{Distill}(\theta_{kb})$ is then estimated by stochastic gradient descent, finding $\theta_{kb}^* = \arg\min_{\theta_{kb}} \mathcal{L}^{Distill}(\theta_{kb})$. This optimization is referred to in Schwarz et al. [13] as online Elastic Weight Consolidation or Online EWC in short. It is important that the penalty can only be applied after the first compress phase. The knowledge base not only acts as a repository of cumulative knowledge but also enables the active column to build upon this consolidated information. The integrated knowledge serves as a foundation for the model to further refine and optimize its learning processes. This iterative learning allows for enhancing the model's performance on subsequent tasks.

Before any subsequent compress phase starts, the retainment of the knowledge base's current parameters θ_{kb}^* is necessary. For doing this, it is important that after the very first compress phase, the diagonal of the Fisher information matrix gets estimated with respect to the latest compressed optimal parameters θ_{kb}^* of the knowledge base network. As the A2C is an on-policy algorithm, in order to calculate the Fisher estimate w.r.t. θ_{kb}^*, we need to generate samples based on the policy $\pi_{kb}(\cdot|x;\theta_{kb}^*)$ to have an accurate estimation of the diagonal of the Fisher information. Estimating the Fisher information is as follows:

$$F(\theta_{kb}^*) = \frac{1}{N}\sum_{t=1}^{N}\left(\frac{\partial(\log(\pi_{kb}(a_t|s_t;\theta_{kb}^*))\cdot A_t)}{\partial\theta_{kb}^*}\right)^2 \tag{A2}$$

where N is the number of samples generated by the policy $\pi_{kb}(\cdot|x;\theta_{kb}^*)$. The algorithmic procedure is outlined in Algorithm 1.

Architecture. An identical architecture is used within both the progress and compress phases. The active column consists of one policy $\pi_{active}(a_t|s_t;\theta_{active}^1)$ and a

Algorithm 1 Distillation + Online EWC Algorithm

Input: Tasks $\{1, 2, \ldots, T\}$, forgetting factor γ, regularization hyperparameter λ
Initialize: Distill (train) Task 1 to obtain $\theta_{kb}^{*(1)} = \arg\min_{\theta_{kb}} \mathcal{L}_1^{Distill}(\theta_{kb})$
for each task $t > 1$ **do**
  Compute FIM: $F_{t-1} = \mathbb{E}_{s_{t-1} \sim P_{t-1}(\cdot)} \left[\left(\frac{\partial(\log(\pi_{kb}(a_{t-1}|s_{t-1};\theta_{kb}^{*(t-1)})) \cdot A_{t-1})}{\partial \theta_{kb}^{*(t-1)}} \right)^2 \right]$
  Online FIM Update: $F_{t-1} = \gamma F_{t-2} + F_{t-1}$
  Distillation Loss: $\mathcal{L}_t^{Distill}(\theta_{kb}) = \mathcal{L}_t(\theta_{kb}) + \frac{\lambda}{2} F_{t-1} \left\| \theta_{kb} - \theta_{kb}^{*(t-1)} \right\|_2^2$
  Update Parameters: $\theta_{kb}^{*(t)} = \arg\min_{\theta_{kb}} \mathcal{L}_t^{Distill}(\theta_{kb})$
end for

value function $V_{active}(s_t; \theta_{active}^2)$ which share a convolutional encoder $\phi_{active}(\cdot; \theta_{active}^3)$. This encoder is represented by a convolutional neural network (CNN), following the approach outlined by Mnih et al. [37]. Let s_t represent the sequence of observations at timestep t from the environment, where $\phi_{active}(s_t)$ indicates the encoded visual features used for subsequent policy learning. The same principle applies to the knowledge base network, with the only difference being the change in the subscript to kb. It is important to note that the active column incorporates lateral connections from the knowledge base. This is done through the function $f(\cdot)$ represented by a neural network consisting of convolutional layers and a multi-layer perceptron with one hidden layer and designed to process the intermediate outputs from the knowledge base. These intermediate outputs are derived from $\phi_{kb}(s_t)$, $\pi_{kb}(a_t|s_t; \theta_{kb}^1)$, and $V_{kb}(s_t; \theta_{kb}^2)$. Initially, during the training of the first task for a specified number of timesteps, the active column does not utilize any lateral connections, keeping the knowledge base inactive. However, after the compress phase, the lateral connections are activated, utilizing prior knowledge to optimize both the active column and the lateral connections in the progress phase of the next task.

Learning in the Progress and Compress Framework. Algorithm 2 represents the approach of Schwarz et al. [13] in the RL domain, unifying all arguments mentioned in the previous sections. In every iteration, a vectorized environment is constructed corresponding to each task.